UNIVERSITY OF UDINE

DEPARTMENT OF MATHEMATICS, COMPUTER SCIENCE AND PHYSICS

READERSOURCING 2.0: DOCUMENTATION

MICHAEL SOPRANO AND STEFANO MIZZARO

v2.0.0

Contents

List of Figures

List of Tables

1 Introduction

The Readersourcing 2.0 ecosystem was built within a research project co-funded by SISSA Medialab[1] and the University of Udine.[2] It was presented by Soprano et al. [5] during the IRCDL 2019 Conference.[3] The original paper is also available on Zenodo.[4] The code and related documentation are available on GitHub.[5]

This version of the documentation describes the modernized 2.0 software stack while preserving the original domain concepts, equations, object-oriented responsibilities, and public communication contracts. It distinguishes the deployed application from the experimental implementations used to study the model. Later evaluations based on simulations and network analysis are discussed by Soprano et al. [10] and Soprano et al. [11].

Initially, a recap of the general architecture is presented, followed by a description of the role and purpose of each component. Specific aspects, such as the technology used, the internal architecture, and the structure of the database, are also discussed. The diagrams adhere to UML where specified and follow the notation guidelines proposed by Fowler [2].

2 General Architecture

Readersourcing 2.0 is composed of several cooperating components. RS_Server [9] (Section 3) gathers ratings and exposes both the application workflows and their RESTful interfaces. RS_Rate [8] (Section 5) is the browser client used by readers. RS_PDF [6] (Section 4) is a command-line component, distributed as a self-contained shaded JAR and bundled with RS_Server, which annotates publications with a rating URL and QR code. Finally, RS_Py (Section 6) provides notebook-based implementations and experiments related to the RSM and TRM models described by Soprano et al. [5].

Figure 1 provides an overview of the architecture of Readersourcing 2.0, and in the following, we briefly describe these four components. We summarize the capabilities of the ecosystem in Section 7.

3 RS_Server

RS_Server [9] is the server-side component that collects and aggregates ratings and uses the RSM and TRM models described by Soprano et al. [5] to compute quality scores for readers and publications. A compatible RS_PDF executable is shipped in the `lib` directory and is invoked as a local process when a publication must be annotated. There may be up

[1] `https://medialab.sissa.it/`
[2] `https://www.uniud.it/`
[3] `https://ircdl2019.isti.cnr.it/`
[4] `https://zenodo.org/record/1446468`
[5] `https://github.com/Miccighel/Readersourcing-2.0`

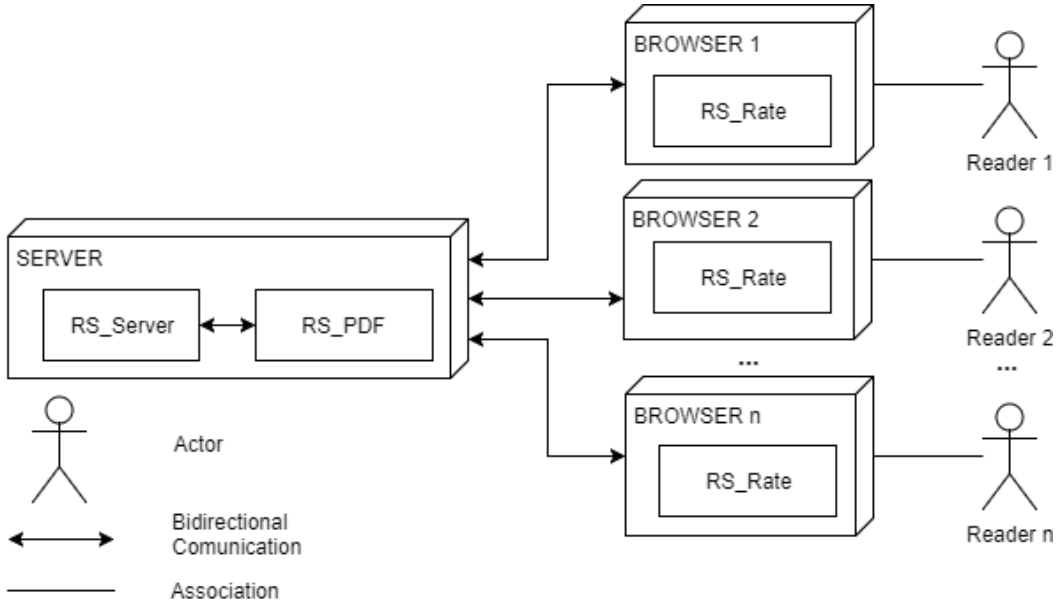

Figure 1: Conceptual architecture of Readersourcing 2.0 (NOT UML). RS_PDF is bundled with and invoked by RS_Server.

to n browsers and end-users communicating with the server, each through an instance of RS_Rate. The current extension distribution targets Chromium-based browsers and Firefox.

This setup facilitates interactions between readers and the server through clients installed on readers' browsers or by utilizing the stand-alone web interface provided by RS_Server. These clients are responsible for managing the registration and authentication of readers, handling rating actions, and managing the download actions of link-annotated publications.

3.1 Scientific Model and Deployed Application

The scientific model, the simulation code, and the deployed application share the same Readersourcing vocabulary but have different scopes. The deployed RS_Server preserves the original three persistent entities—users, publications, and ratings—and the RSM/TRM update order implemented by the original application. Experimental repositories may introduce additional entities, such as explicit authors, or evaluate alternative simulation assumptions without implicitly changing the production domain.

The modernization work therefore treats the domain behavior as a compatibility boundary. Framework versions, build systems, browser manifests, and deployment tooling may change, while the strategy abstraction, score computation order, database tables, routes, and JSON contracts remain stable unless a separate domain change is explicitly designed and tested.

During the design phase of RS_Server, strategies were adopted to ensure extensibility

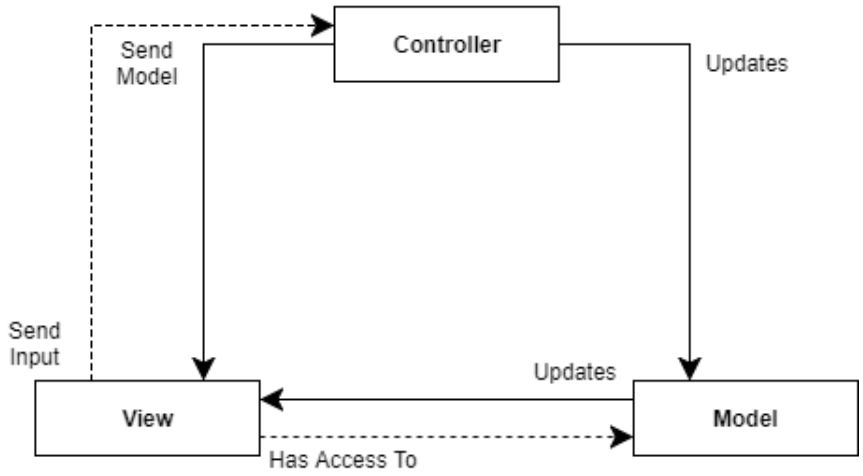

Figure 2: Intuitive scheme of the MVC pattern (NOT UML).

and generality. This includes:

(i) Straightforward addition of new models.

(ii) A shared input data format among all models.

(iii) A standard procedure for models needing to save values locally in the RS_Server database.

3.2 Implementation and Technology

RS_Server is developed in Ruby on Rails,[6] which is a framework that allows building applications strongly based on the Model-View-Controller (MVC) architectural pattern. The technology used to develop RS_Server is an open-source web application framework called *Ruby on Rails* (also referred to as *RoR* or *Rails* only). More specifically, *Rails* is the framework built above *Ruby*, the actual programming language. It allows building applications strongly based on the Model-View-Controller (MVC) architectural pattern.

The MVC pattern allows for the separation of the control logic of the program from data presentation and business logic. Therefore, it facilitates the creation of an effective architecture from the initial design phase. Figure 2 provides an intuitive representation of the structure that this pattern enables. The structure comprises three distinct entities: the *Controller*, responsible for managing control logic; the *Model*, tasked with encapsulating business logic; and the *View*, responsible for implementing data presentation.

The Controller has direct access to both the Model and the View. Typically, it receives user input from the View and, based on this input, updates the internal state of the Model using its methods. Subsequently, the Controller sends the updated Model to the View,

[6] `https://rubyonrails.org/`

which then utilizes this information to obtain and display the results of the processing. In a generic software system, there can be more than one Controller, and each Controller may manage multiple instances of the Model. In MVC frameworks designed for web applications, such as Rails, it is common practice to have a number of Controllers equal to the entities modeled within the application domain. Additionally, there may be more than one View implementation to present the internal state of a specific type of Model.

The use of the MVC pattern is not the sole foundational principle of Rails. One of the most crucial principles shaping Rails for the development of high-quality applications is "Convention Over Configuration". In essence, the framework aims to reduce the decisions developers need to make during application construction by embracing standard conventions that can be modified for increased flexibility if necessary. To delve deeper into the foundational principles of Rails, one can refer to the "Rails Doctrine".[7]

As a final note, Rails is a dynamically evolving framework widely employed in the industry by prominent players such as *GitHub*, *SoundCloud*, and others. It is a widely embraced and esteemed technology with an active community and abundant learning resources.

3.3 Communication Paradigm

A modern MVC framework like Rails provides the capability to develop various types of web applications. One such possibility is the creation of a *Web Service*, which is a software component capable of executing various operations made remotely accessible through the exchange of messages encoded in a standard interchange format such as *JSON*. This is facilitated by a transport layer built on top of basic Internet protocols like *HTTP*. However, the implementation of this functionality must adhere to a paradigm that precisely defines the available functionalities (resources and operations) and specifies the required messages for accessing them.

One of the communication paradigms for Web Services is *RESTful* (*REpresentational State Transfer*). In this paradigm, the functionalities of a Web Service are represented by resources identified through different URIs, and the type of HTTP message sent determines the operation to be performed. The result of the operation initiated by the received message from the Web Service is a new message encoded in the same interchange format as the one sent. It is the client's responsibility to accurately interpret and utilize the response from the Web Service.

RS_Server is a Rails application exposing RESTful interfaces alongside server-rendered web workflows. API clients exchange JSON messages over HTTP, while the same domain and authorization rules are reused by the web interface.

The implementation routes and integration tests are the authoritative communication contract. A historical Postman collection with request examples remains available at

`https://documenter.getpostman.com/view/4632696/RWTiwfV4?version=latest`.

[7] `https://rubyonrails.org/doctrine/`

Examples in the external collection should be checked against the version of RS_Server being deployed.

To illustrate, Table 1 presents a subset of the RESTful interface of RS_Server. These operations encompass all functionalities related to managing one of the entities within the application domain, specifically, publications. Suppose a user initiates a "show" operation for a publication with an identifier equal to 1 by accessing the corresponding endpoint. In response, RS_Server would provide a JSON-encoded response similar to the example below.

```json
{
  "id": 1,
  "doi": "10.1140/epjc/s10052-018-6047-y",
  "title": "Uncertainties in WIMP dark matter scattering revisited",
  "subject": null,
  "author": "John Ellis",
  "creator": "Springer",
  "producer": null,
  "pdf_url": "https://example.org/publication.pdf",
  "steadiness": 1.0,
  "score_rsm": 0.71,
  "score_trm": 0.68,
  "created_at": "2026-01-01T12:00:00.000Z",
  "updated_at": "2026-01-01T12:00:00.000Z",
  "url": "http://localhost:3000/publications/1.json"
}
```

3.4 Database

To facilitate the implementation of functionalities such as the storage of link-annotated publications and user authentication, a database schema has been defined, as illustrated in Figure 3. Within the application domain of Readersourcing 2.0, three main entities have been modeled:

- **Users**: models the users of the system, characterized by their personal data. Users may have an optional *ORCID* and a boolean indicating whether they wish to receive emails. Additionally, various attributes are utilized to store different types of tokens, enabling operations such as password reset.
- **Ratings**: represents the ratings provided by readers for publications. These ratings are characterized by a numerical score. The entity also includes a boolean to indicate whether a rating is anonymous, and another boolean to track whether it has been edited at a later time.

Endpoint	HTTP Message	Operation	Description
`/publications.json`	GET	Index	Fetches the entire collection of Publications.
`/publications/list`	GET	List	Fetches the entire collection of Publications (server-side view).
`/publications/1.json`	GET	Show	Returns the Publication with identifier equal to 1.
`/publications/lookup.json`	POST	Lookup	Searches for a Publication; if it does not exist, it is fetched from the given URL.
`/publications/random.json`	GET	Random	Returns a random Publication.
`/publications/1/is_rated.json`	GET	Is Rated	Checks if the Publication with identifier equal to 1 has been rated by at least one reader.
`/publications/1/is_saved_for_later.json`	GET	Is Saved For Later	Checks if the Publication with identifier equal to 1 has been saved for later by the current user.
`/publications.json`	POST	Create	Creates a new Publication.
`/publications/is_fetchable.json`	POST	Is Fetchable	Checks if the provided URL contains a fetchable Publication.
`/publications/extract.json`	POST	Extract	Extracts the rating URL from a link-annotated Publication.
`/publications/fetch.json`	POST	Fetch	Fetches a Publication from the given URL.
`/publications/1/refresh.json`	GET	Refresh	Fetches the Publication with identifier equal to 1 again.
`/publications/1.json`	PUT	Update	Updates the Publication with identifier equal to 1.
`/publications/1.json`	DELETE	Delete	Deletes the Publication with identifier equal to 1.
...	...	...	...

Table 1: Subset of the RESTful interface of RS_Server.

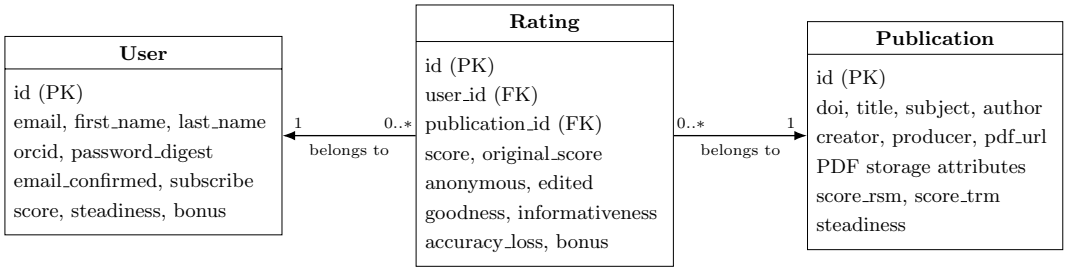

Figure 3: Current persistent entities and associations in the RS_Server database (selected attributes).

- **Publications**: models publications known to the server, including publications saved for later but not yet rated. They are characterized by an optional *DOI*, descriptive metadata, and attributes used to manage the original and link-annotated PDF files.

In addition, each of these entities is further characterized by additional attributes (*steadiness*, *informativeness*, ...), representing the scores or parameters computed by the Readersourcing models.

Figure 3 depicts the two mandatory associations of a rating. A user may give zero or more ratings, but each rating belongs to exactly one user. An anonymous rating still retains this association so that authentication and the one-rating-per-user constraint can be enforced; anonymity controls how the reader identity participates in presentation and processing, not referential integrity.

Similarly, a publication may receive zero or more ratings, while each rating belongs to exactly one publication. The zero multiplicity is required because a publication may be fetched and saved for later before any rating is submitted.

3.5 Class Diagram

Figure 4 shows the main responsibilities of the current RS_Server implementation. Rails controllers map HTTP operations to the domain models, while JSON and server-rendered views present their results. The three persistent models retain the business logic and associations described in the previous section. Authentication and Readersourcing computations are delegated to focused command and strategy objects rather than being absorbed into the controllers.

The *ApplicationController* contains shared authorization hooks for both API and server-side requests, while the *AuthenticationController* handles login and logout. The *Authenticator* and *Authorizer* command objects isolate credential verification, token issuance, and token validation from HTTP concerns.

Due to this design choice, the traditional server-side approach to user authentication, where information about the logged user is stored in session data, is not viable. This is because the RESTful paradigm is *stateless*. To authenticate themselves, users must attach a

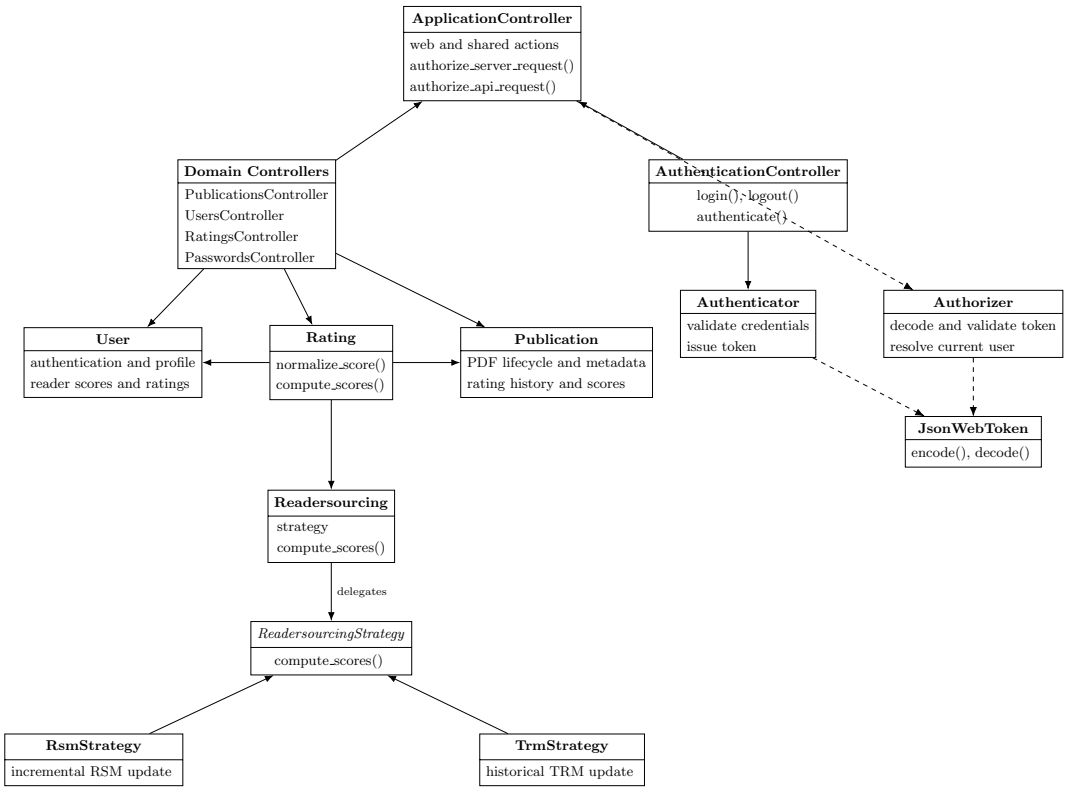

Figure 4: High-level class diagram of the modernized RS_Server. Method lists are intentionally selective.

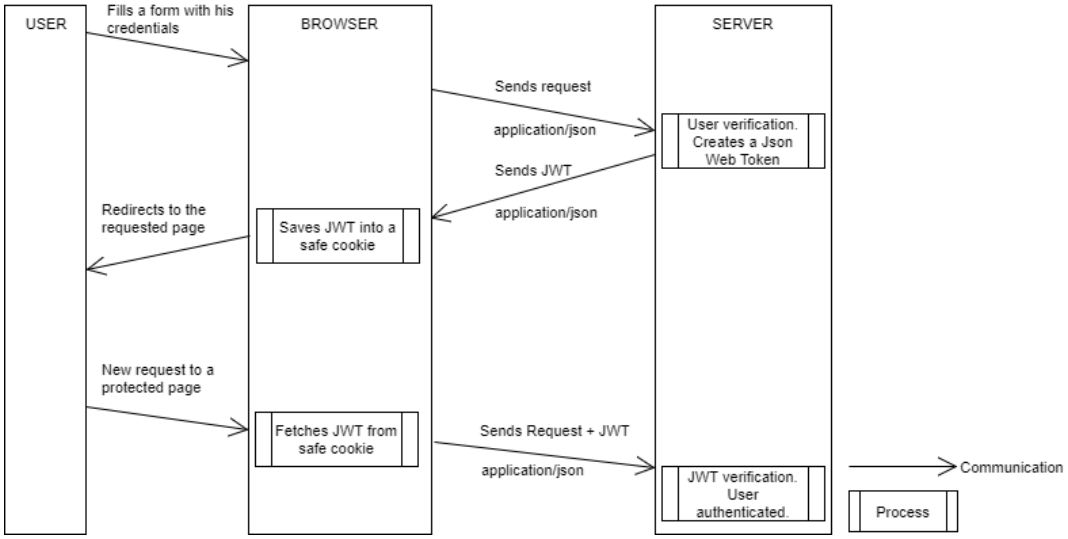

Figure 5: Representation of the token-based authentication process (NOT UML).

token to each request, identifying their validity within the system. Therefore, a *token-based* authentication approach has been implemented.

When a user logs in, the supplied credentials are verified against the stored password digest and the account confirmation state. The credentials themselves are not placed in the token. If verification succeeds, RS_Server issues a signed JWT whose payload contains the user identifier, the client IP address, and an expiration time.

The browser client stores the returned token in its application cookie and attaches it to subsequent API requests through the `Authorization` header. The server-rendered workflow also keeps a duplicate token in the Rails session, which is implemented through an HTTP-only cookie. Upon receiving a protected request, the *Authorizer* verifies the JWT signature, expiration time, IP address, and referenced user before allowing the request to proceed. Figure 5 provides an intuitive illustration of this authentication process.

The classes responsible for the Readersourcing computations continue to follow the *Strategy* pattern: *Readersourcing*, *ReadersourcingStrategy*, *RsmStrategy*, and *TrmStrategy*. A rating invokes the RSM strategy first and the TRM strategy second, preserving the original update order and quantities. The pattern keeps model selection extensible without weakening the fidelity of the existing domain behavior.

3.6 Deploy

The current codebase supports two verified setup paths: a manual installation, intended primarily for development, and a Docker Compose deployment, which reproduces the production stack locally. The modernized application was verified with both paths while preserving the existing routes, database schema, JSON contracts, and Readersourcing strate-

gies.

Environment-specific secrets and database settings are described in Section 3.6.4. A local `.env` file must never be committed.

3.6.1 Modality 1: Manual

This modality downloads and initializes RS_Server directly on the local machine and offers the greatest flexibility for development and domain-model experiments.

3.6.1.1 Requirements

- Ruby == 3.4.10;
- Bundler compatible with the committed `Gemfile.lock`;
- a Java 21 runtime, required by RS_PDF;
- PostgreSQL >= 17;
- Node.js >= 18, used to install and precompile the existing browser assets.

3.6.1.2 How To

Clone the RS_Server repository,[8] navigate to its root directory, create the `.env` file described in Section 3.6.5, and run:

```
bundle install
node .yarn/releases/yarn-3.6.3.cjs install --immutable
bin/rails db:create
bin/rails db:migrate
bin/rails db:seed  # optional sample data
bin/rails server -b 127.0.0.1 -p 3000 -e development
```

PostgreSQL must be accepting connections before the database commands are executed. With the sample bind address and port, the application is available at `http://127.0.0.1:3000`. Rails binstubs are always invoked from the repository root so that the locked application dependencies are used.

3.6.1.3 Quick Cheatsheet

1. `cd` to main directory;
2. create and populate the `.env` file;
3. `bundle install`;
4. `node .yarn/releases/yarn-3.6.3.cjs install --immutable`;

[8] `https://github.com/Miccighel/Readersourcing-2.0-RS_Server`

5. `bin/rails db:create`;
6. `bin/rails db:migrate`;
7. `bin/rails db:seed` (optional);
8. `bin/rails server -b <address> -p <port> -e development`.

3.6.2 Modality 2: Docker Compose

Docker Compose builds RS_Server from the current checkout and starts it together with PostgreSQL 17. The application image includes Ruby 3.4.10, the Java 21 runtime required by RS_PDF, the locked Ruby dependencies, and the precompiled browser assets. This is the recommended reproducible deployment for local integration and production-like verification.

3.6.2.1 Requirements

- Docker Desktop or another installation supporting Docker Compose v2.[9]

3.6.2.2 How To

Clone the repository, create the `.env` file, ensure that the Docker Engine is running, and execute:

```
docker compose up --build
```

The `database` service must pass its health check before the `rs_server_webapp` service starts. The application entrypoint runs `bin/rails db:create` and `bin/rails db:migrate` automatically. Once startup completes, the web application is available at `http://localhost:3000`; `0.0.0.0` is the container bind address, not the address users should enter in a browser.

Sample data remain opt-in:

```
docker compose run --rm rs_server_webapp bin/rails db:seed
```

Use `docker compose down` to stop the services. The named PostgreSQL volume is retained so that application data survive ordinary container recreation.

Quick Cheatsheet

- `cd` to main directory;
- create and populate the `.env` file;
- `docker compose up --build`;
- `docker compose run --rm rs_server_webapp bin/rails db:seed` (optional);
- `docker compose down` (to stop the services).

[9] `https://docs.docker.com/compose/`

3.6.3 Historical Heroku Deployment

Earlier versions of this document described a Heroku Container Registry deployment. That procedure has not been revalidated against Rails 8, the current Java 21 runtime, or current Heroku product requirements, and is therefore retained only as historical context. Docker Compose is the verified production-like deployment path. Before restoring Heroku or another PaaS as a supported target, its release phase, database migration strategy, persistent-file requirements, SMTP configuration, and health checks must be tested explicitly.

3.6.4 Environment Variables

Deployment-specific credentials and connection details are supplied through environment variables. Table 2 lists the variables used by the current application. Database URLs take precedence over individual PostgreSQL settings.

Environment Variable	**Purpose**	**Scope**
`SECRET_DEV_KEY`	Rails secret used in development.	development
`SECRET_PROD_KEY`	Rails secret used to sign and encrypt production data, including JWTs and session data.	production
`DATABASE_URL`	Complete PostgreSQL connection URL. Overrides the individual `POSTGRES_*` variables.	development, production
`TEST_DATABASE_URL`	Complete PostgreSQL connection URL for the test database.	test
`POSTGRES_USER`	PostgreSQL user name used when a full URL is not supplied.	all database environments
`POSTGRES_PASSWORD`	Password for the PostgreSQL user.	all database environments
`POSTGRES_HOST`	PostgreSQL host; defaults to `localhost` outside Compose and is set to `database` by Compose.	all database environments
`POSTGRES_DB`	Development or production database name; defaults to `rs_server`.	development, production
`POSTGRES_TEST_DB`	Test database name; defaults to `rs_server_test`.	test
`SMTP_USERNAME`, `SMTP_PASSWORD`	Credentials for the configured SMTP service.	production mail

Environment Variable	Purpose	Scope
SMTP_DOMAIN_NAME, SMTP_DOMAIN_ADDRESS	Sender domain and SMTP server address.	production mail
EMAIL_BUG_REPORT, EMAIL_ADMIN	Recipient addresses for reports and general contact messages.	development, production
RAILS_LOG_TO_STDOUT	When present, sends production Rails logs to standard output.	production
RAILS_MAX_THREADS	Maximum Active Record connection pool size; defaults to 5.	all environments

Table 2: Environment variables of RS_Server.

3.6.5 Setting Variables

For local and Compose deployments, create a `.env` file in the RS_Server root and populate it using `key=value` entries. Do not quote secrets unless the selected environment loader requires it, and never commit the file.

To provide an example, the following is the content of a valid `.env` file.

```
SECRET_PROD_KEY=replace-with-a-long-random-secret
POSTGRES_USER=postgres
POSTGRES_PASSWORD=replace-with-a-database-password
POSTGRES_DB=rs_server
SMTP_USERNAME=replace-with-an-smtp-user
SMTP_PASSWORD=replace-with-an-smtp-password
SMTP_DOMAIN_NAME=readersourcing.example
SMTP_DOMAIN_ADDRESS=smtp.example
EMAIL_BUG_REPORT=bugs@readersourcing.example
EMAIL_ADMIN=contact@readersourcing.example
RAILS_LOG_TO_STDOUT=true
```

3.6.6 Sending Mails

RS_Server supports standard SMTP services for account confirmation, password recovery, bug reports, and contact messages. The SMTP provider must expose a host, port 587 with STARTTLS, a user name, a password, and a verified sender domain. Provider-specific API keys can be placed in `SMTP_PASSWORD` when the provider uses an API key as its SMTP credential; the provider's current documentation remains authoritative for those values.

3.6.7 Connecting To The Database

A full connection string supplied through `DATABASE_URL` takes precedence over the individual `POSTGRES_*` variables in development and production. The test environment follows the same rule using `TEST_DATABASE_URL` and `POSTGRES_TEST_DB`.

```
ENV['DATABASE_URL'] ||
  "postgresql://#{ENV['POSTGRES_USER'] || 'postgres'}:" \
  "#{ENV['POSTGRES_PASSWORD']}@" \
  "#{ENV['POSTGRES_HOST'] || 'localhost'}/" \
  "#{ENV['POSTGRES_DB'] || 'rs_server'}"
```

3.6.8 Logging To The Standard Output

Development writes logs to standard output by default. In production, setting `RAILS_LOG_TO_STDOUT` configures a tagged logger on standard output, which is appropriate for Docker and other distributed deployments.

4 RS_PDF

RS_PDF [6] is the command-line component used by RS_Server to edit PDF files when a reader requests to save a publication for later reading. It appends a final page containing a clickable rating URL and a QR code, and stores the same URL in the PDF `BaseUrl` metadata field.

The executable is distributed as the self-contained `RS_PDF-v2.0.0-shaded.jar` and bundled in the RS_Server `lib` directory. RS_Server invokes it as a local Java process, so no separate network service or protocol is required between the two components.

4.1 Implementation and Technology

The technology used to develop RS_PDF is the Kotlin object-oriented programming language, known for its compatibility with the Java Virtual Machine. The current build targets Java 21 and uses Kotlin 2.4, Apache PDFBox 3 for PDF manipulation, Apache Commons CLI for argument parsing, Log4j 2 for logging, and ZXing for QR-code generation.

Kotlin was chosen because it provides modern object-oriented language features while interoperating directly with Java libraries. The underlying PDF toolkit is *PDFBox*,[10] while ZXing[11] produces the QR code placed beside the link annotation.

Kotlin has been created by JetBrains,[12] which, in the first half of 2017, signed an agreement with Google to elevate Kotlin to the status of a first-class language for development on the Android platform.[13] Moreover, in the same year, JetBrains announced the ability to

[10] `https://pdfbox.apache.org/`
[11] `https://github.com/zxing/zxing`
[12] `https://www.jetbrains.com/`
[13] `https://blog.jetbrains.com/kotlin/2017/05/kotlin-on-android-now-official/`

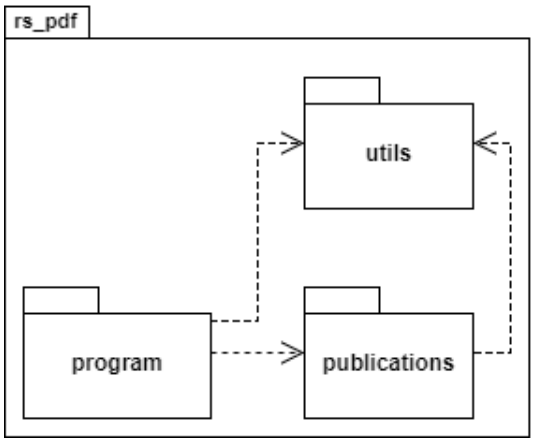

Figure 6: Package diagram of RS_PDF.

compile programs written in Kotlin directly into machine language, thus avoiding the use of the JVM.

On the web, it is possible to find different pages with comparisons between Kotlin and other languages, including the official one[14] made by JetBrains with Java, and several articles[15] by developers enthusiastic about this programming language.

4.2 Package Diagram

Figure 6 depicts a diagram illustrating the packages in which RS_PDF is divided. This diagram offers a high-level overview of the internal architecture of the software, providing valuable insights into its structure.

In particular, the interaction with RS_Server begins in the **program** package. The *Program* object parses and validates the command-line options before delegating to the publication controller.

The **utils** package provides shared filesystem defaults, program constants, and logging configuration. Server addresses are supplied through the required URL argument and are not hard-coded as constants.

The **publications** package contains the PDF business logic. Its Controller/Model separation follows the same object-oriented intent as MVC without depending on Rails: the controller discovers one or more PDF files and creates the corresponding models, while each model loads a document, adds the link and QR code, writes metadata, and saves a new file. A View is unnecessary because the result is the edited PDF itself.

[14] `https://kotlinlang.org/docs/reference/comparison-to-java.html`
[15] `https://medium.com/@octskyward/why-kotlin-is-my-next-programming-language-c25c001e26e3`

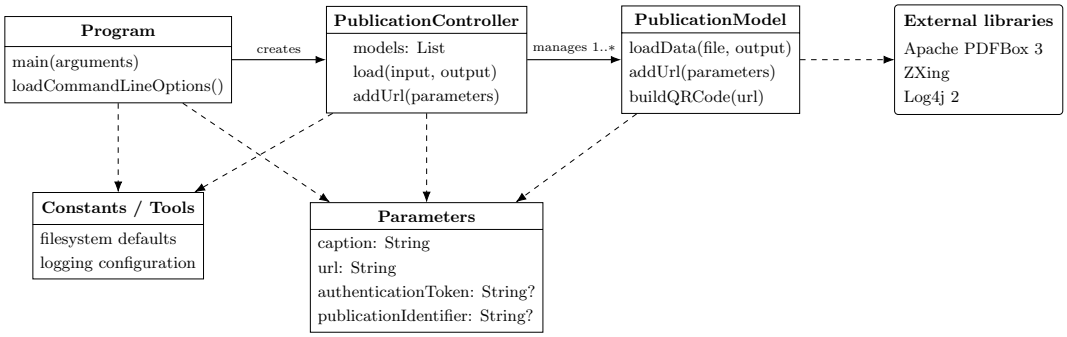

Figure 7: High-level class diagram of RS_PDF v2.0.0.

4.3 Class Diagram

Figure 7 shows the main classes of RS_PDF. The structure deliberately preserves the original Controller/Model responsibilities: *Program* owns process-level parsing and flow, *PublicationController* manages the collection, and *PublicationModel* performs PDF transformations through PDFBox and ZXing.

The *Parameters* data class transports the caption, URL, optional authentication token, and optional publication identifier through the controller boundary. This keeps the *PublicationModel* interface stable as related input data evolve.

4.4 Installation

RS_PDF is bundled with compatible RS_Server distributions, eliminating the need for a separate installation. To use it independently, download the shaded JAR from the repository releases[16] or build it from source:

```
./mvnw clean verify
```

On Windows use `mvnw.cmd clean verify`. The Maven Wrapper selects the configured Maven version and the executable is generated as `target/RS_PDF-v2.0.0-shaded.jar`.

4.4.1 Requirements

- Java runtime >= 21 to execute the shaded JAR;
- JDK >= 21 to compile and verify the project from source.

4.4.2 Command Line Interface

The behavior of RS_PDF is configured through the command-line options in Table 3. Caption and URL are always required. Input and output paths may be omitted only together, in which case the default `data` and `res` directories are used. The authentication

[16] `https://github.com/Miccighel/Readersourcing-2.0-RS_PDF`

token and publication identifier are optional, but if either is supplied then both custom paths and both server values must be supplied.

Short	Long	Description	Constraint
`-pIn`	`--pathIn`	Input PDF file or directory, using a relative or absolute path.	Optional; requires `-pOut`.
`-pOut`	`--pathOut`	Existing output directory for annotated PDF files.	Optional; requires `-pIn`.
`-c`	`--caption`	Caption of the clickable link added to the PDF.	Required.
`-u`	`--url`	Rating URL used by the link, QR code, and `BaseUrl` metadata.	Required valid URL.
`-a`	`--authToken`	Authentication token received from RS_Server.	Optional; requires paths and `-pId`.
`-pId`	`--publicationId`	Publication identifier received from RS_Server.	Optional; requires paths and `-a`.

Table 3: Command line options of RS_PDF.

To provide an execution example, let's assume a scenario where there is a need to edit some files encoded in PDF format with the following prerequisites:

- there is a folder containing n PDF files at `C:\data`;
- annotated files must be saved in the existing folder `C:\out`;
- the shaded JAR is located at `C:\lib\RS_PDF-v2.0.0-shaded.jar`.

The execution of RS_PDF is started with the following command:

```
java -jar C:\lib\RS_PDF-v2.0.0-shaded.jar \
  -pIn C:\data -pOut C:\out \
  -c "Express your rating" \
  -u "https://readersourcing.example/rate"
```

5 RS_Rate

RS_Rate [8] is a browser extension that acts as a client for the Readersourcing 2.0 ecosystem without requiring users to navigate to its website first. The current Manifest V3 codebase produces separate self-contained builds for Chromium-based browsers, including Google

Chrome[17] and Microsoft Edge,[18] and for Mozilla Firefox.[19]

The primary objective of RS_Rate is to provide readers with a way to seamlessly rate a publication with minimal effort—just a few clicks or keystrokes within the Readersourcing 2.0 ecosystem, contributing to a more dynamic online reading experience. RS_Rate serves as the initial client of our project, extending beyond the web-based interface available on the main portal.

The two targets share the same application source and domain workflow. Browser-specific manifests and build output isolate platform requirements without duplicating the rating logic. Safari and other browser families remain possible future targets.

5.1 Implementation and Technology

RS_Rate is developed with standard web technologies: HTML, CSS, and JavaScript. Its dependencies remain declared in `package.json` and resolved through the committed Yarn lockfile. The build process copies and checks all executable assets locally, then produces unpacked Manifest V3 extensions in `dist/chromium` and `dist/firefox`. Store archives are generated from those outputs rather than treating CRX as the only distribution format.

The JavaScript implementation uses jQuery for DOM manipulation and asynchronous API requests. Bootstrap 4 and its jQuery-based plugins are intentionally retained because the existing views depend on their markup and integration behavior; moving to Bootstrap 5 is a separate user-interface migration rather than a dependency-only update.

5.2 Installation

RS_Rate is freely available on the Google Chrome Web Store. The Chromium release can be installed from the following page:

- **Google Chrome and Microsoft Edge**: `https://chromewebstore.google.com/detail/readersourcing-20-rsrate/hlkdlngpijhdkbdlhmgeemffaoacjagg`

The repository also produces a Firefox Manifest V3 package with the stable Gecko identifier `rs-rate@readersourcing.org`. This package is ready for validation and submission to Mozilla Add-ons, but this statement does not imply that a signed AMO release has already been published. For local testing, load `dist/firefox/manifest.json` from `about:debugging#/runtime/this-firefox` using *Load Temporary Add-on*.

5.3 Development and Packaging

The development toolchain uses Node.js 24 LTS and Yarn 4.17.1. From the repository root:

[17] `https://www.google.com/chrome/`
[18] `https://www.microsoft.com/en-us/edge/`
[19] `https://www.mozilla.org/firefox/`

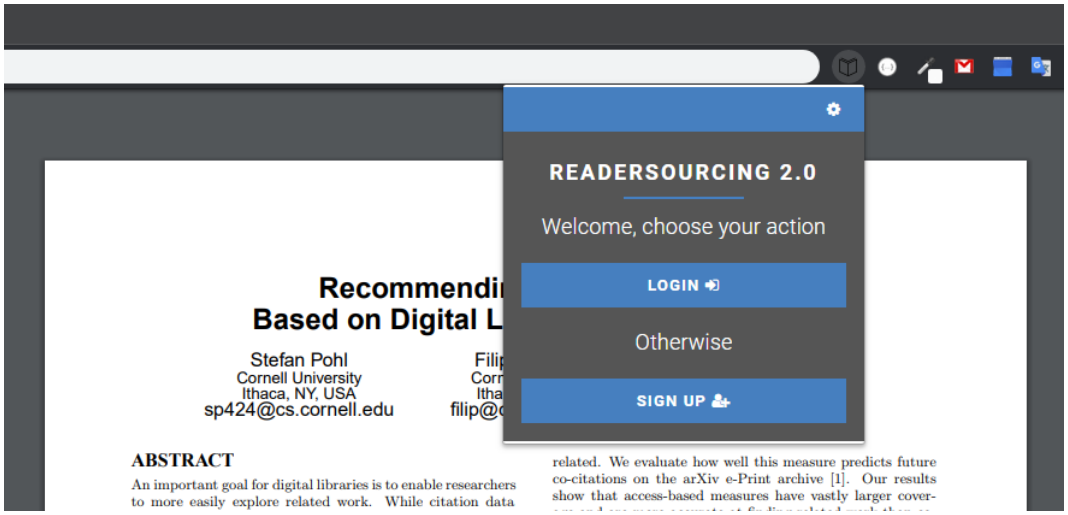

Figure 8: RS_Rate characterized as an extension having a popup action.

```
corepack enable
yarn install --immutable
yarn verify
```

`yarn verify` builds both targets, checks that all runtime assets are local, validates the Firefox build with Mozilla's `web-ext`, and runs the client contract tests. The Firefox archive intended for Mozilla Add-ons is generated with:

```
yarn package:firefox
```

The ZIP is written to `artifacts`. The Firefox target requires Firefox 140 or later. Its manifest declares the account, browsing, website-interaction, and publication-content categories required by the existing Readersourcing workflow; no separate analytics channel is introduced.

Development builds initially target `http://localhost:3000`. Users can change the RS_Server host in the extension options, and the same configured host is used for normal API requests and PDF extraction.

5.4 Usage

Figure 8 illustrates a section of a Google Chrome instance with the extension active for a publication. This scenario depicts the typical situation of a reader visiting a publisher's website to access the PDF of a paper they are interested in. The figure also displays the initial page that a reader encounters when interacting with the client. This page serves as a gateway to the login page, as shown in Figure 9, or to the sign-up page. From the login page, a reader who has forgotten their password can access the password recovery page (not shown), which closely resembles the login page.

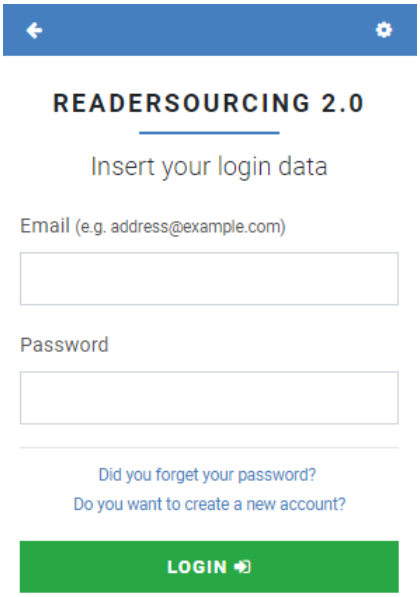

Figure 9: The login page of RS_Rate.

If a reader has yet to sign up for Readersourcing 2.0, they can navigate from the page shown in Figure 8 to the one displayed in Figure 11 and fill in the sign-up form. Once they complete the standard sign-up and login operations, they will find themselves on the rating page, as shown in Figure 10.

In the central section of the rating page, a reader can use the slider to choose a rating value in a 0-100 interval. Once they select the desired rating, they only need to click the green *Rate* button, and that's it; with just three clicks and a slide action, they can submit their rating. Furthermore, they can also click the options button and, if preferred, check an option to anonymize the rating they are about to provide. It's important to note that the reader has to be logged in to express an anonymous rating to prevent spamming, which in this case would be a very dangerous phenomenon. When such a rating is processed, the information regarding its reader will not be used, except for preventing the reader from rating the same publication multiple times.

If the reader prefers to provide their rating at a later time instead of immediately rating the publication, they can click the *Save for later* button. This option allows them to take advantage of the editing procedure for publications, which stores a reference (an URL link) inside the PDF file they are viewing. As soon as such the editing procedure is completed (usually just a few seconds), the *Save for later* button transforms into a *Download* button, as shown in Figure 12.

The reader can finally download the link-annotated publication by clicking on it. Furthermore, they can also use the refresh button (located to the right of the *Download* button) to, as it says, refresh the link-annotated publication. This means that a new copy of the

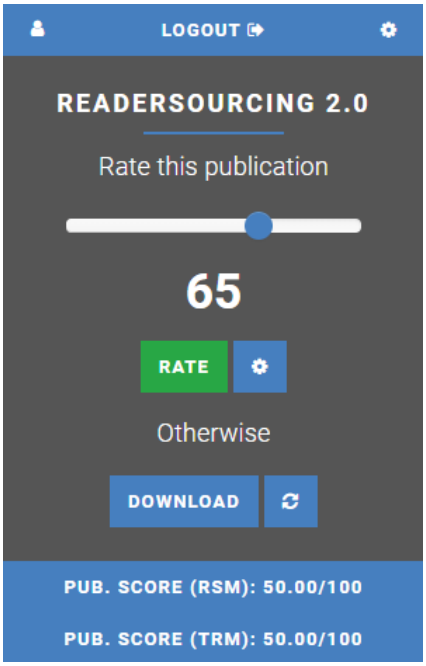

Figure 10: The rating page of RS_Rate.

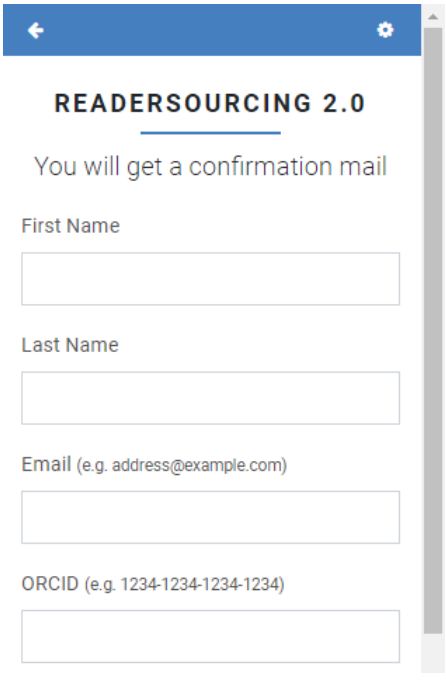

Figure 11: The user registration page of RS_Rate.

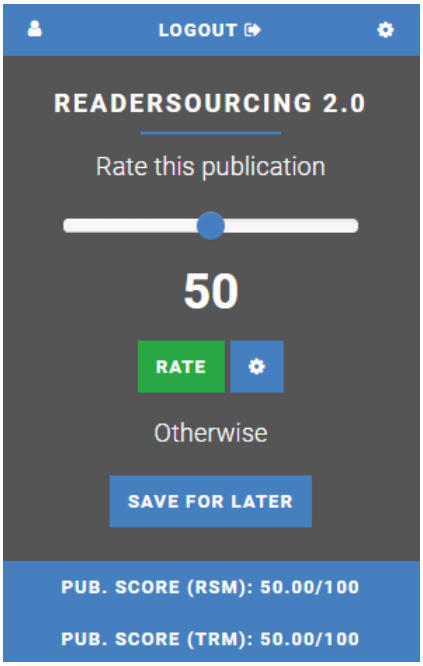

Figure 12: The rating page of RS_Rate after a "save for later" request.

publication file will be downloaded, annotated, and made available to the reader. This feature is useful since a publication could be updated at a later time by its author.

As soon as the link-annotated publication is downloaded, the reader will find a PDF containing a new final page with the URL. In Figure 13, an example of such a link-annotated publication can be seen; in that case, the reader has chosen to open it with their favorite PDF reader.

Once the reader clicks on the reference, which is a special link to RS_Server, they will be taken to the server-side application itself. The interface presented allows them to express their rating independently of the browser extension used to store the reference. Therefore, if they send their link-annotated publication to a tablet-like device, for example, they can take advantage of the built-in browser to express their rating. Figure 14 shows the interface that the reader sees after clicking on the stored reference. The reader is required to authenticate themselves again as a form of security. Without this step, the stored reference could be used by anyone who gets a copy of the link-annotated publication.

Every time a reader rates a publication, every score is updated according to both RSM and TRM models, and each reader can see the result through RS_Rate. In the bottom section of the rating page, the score of the current publication can be seen (one for each model), as shown in Figures 10 and 12. To view their score as a reader (once again, one for each model), a user must click the profile button in the upper right corner. Once they do that, they will see the interface shown in Figure 15. From there, they can also edit their password since that interface acts as a profile page.

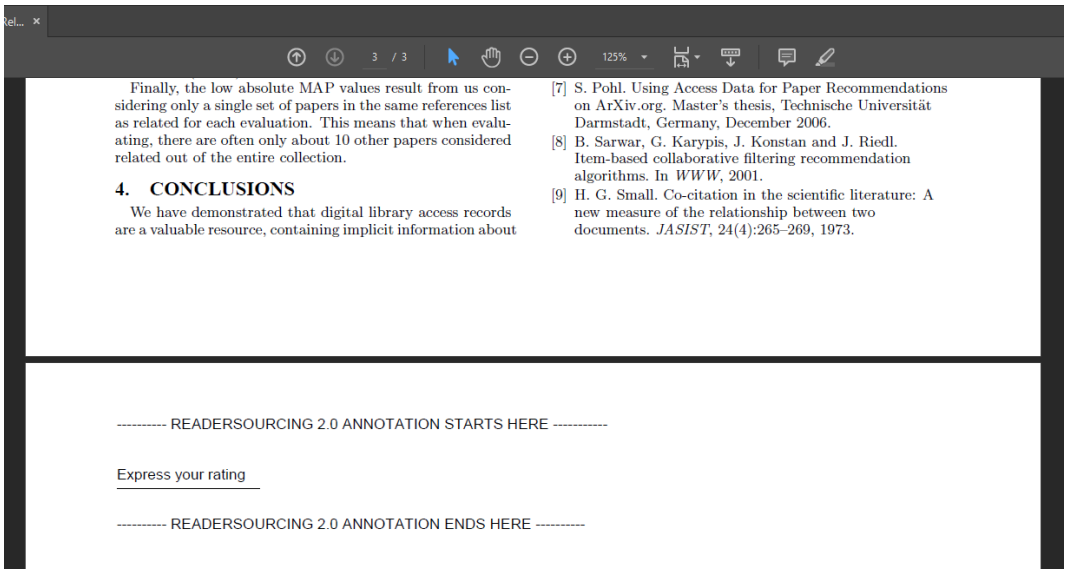

Figure 13: A publication link-annotated through RS_Rate.

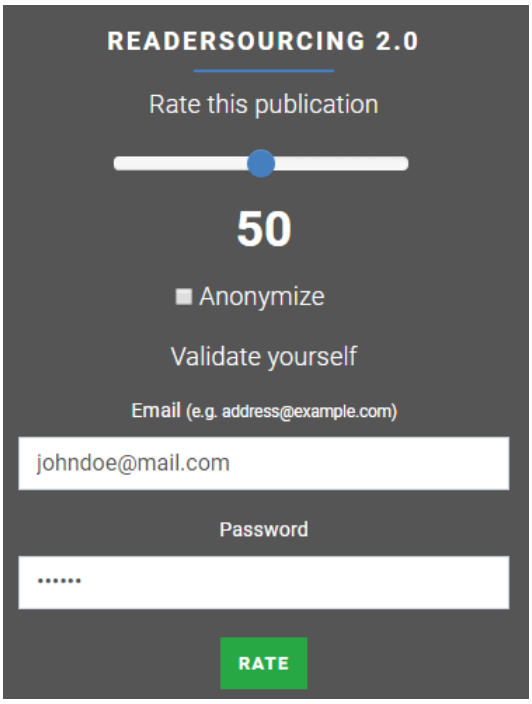

Figure 14: The server-side interface to rate a publication.

Figure 15: The profile page of RS_Rate.

6 RS_Py

RS_Py [7] is an experimental component of the Readersourcing 2.0 ecosystem, providing notebook-based implementations of the models presented by Soprano et al. [5]. The deployed Ruby implementations remain encapsulated by RS_Server as described in Section 3; RS_Py supports exploration, dataset generation, and comparison without changing the server.

Developers with a background in the Python programming language can leverage RS_Py to generate and test new simulations of ratings given by readers to a set of publications. They are allowed to alter the internal logic of the models to test new approaches without the need to fork and edit the full implementation of Readersourcing 2.0.

6.1 Implementation and Technology

RS_Py is a collection of *Jupyter Notebooks*. Jupyter[20] is a Python-powered[21] open-source web application that allows the creation and sharing of documents containing live code, equations, visualizations, and narrative text. Notebooks can be shared with others and are composed of cells that can be run independently, providing a step-by-step overview of the implemented computations.

[20] `https://jupyter.org/`
[21] `https://www.python.org/`

6.2 Installation

Clone the RS_Py repository,[22] create an isolated Python environment compatible with its current requirements, and install the packages declared in `requirements.txt`:

```
python -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
jupyter notebook
```

On Windows, activate the environment with `.venv\Scripts\activate`. The repository currently declares its scientific Python packages without exact version pins, so results intended for publication should record the resolved environment separately.

6.3 Usage

RS_Py is organized into the following tracked areas:

- `data` stores input datasets and generated archives;
- `models` stores serialized model outputs and computed quantities;
- `notebooks` contains dataset generation, model implementation, and experiment notebooks;
- `src` contains legacy conversion support;
- `utils` contains auxiliary tooling, including the power-law generator.

Within the tracked `notebooks` folder, the principal files are:

- `1.Seed-Power-Law.ipynb` generates synthetic data using power-law distributions;
- `2.Readersourcing.ipynb` implements and runs the RSM model;
- `3.TrueReview.ipynb` implements and runs the TRM model;
- `4.Experiments.ipynb` supports experimental analysis;
- `ReadersourcingToolkit.py` contains shared Python functionality used by the notebooks.

The `src/Convert.py` helper still refers to historical, unnumbered notebook names and a generated `scripts` directory that is not part of the current tracked tree. It must therefore be treated as legacy tooling until it is realigned and verified. The notebooks themselves remain the authoritative executable artifacts for RS_Py.

[22] `https://github.com/Miccighel/Readersourcing-2.0-RS_Py`

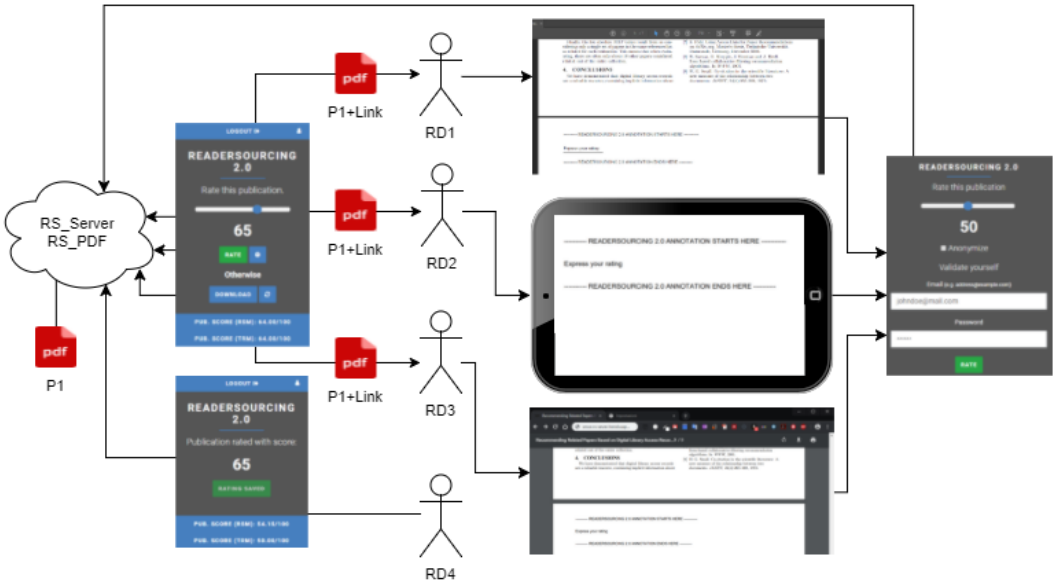

Figure 16: Readers' interaction modalities with the Readersourcing 2.0 ecosystem.

Start Jupyter from the repository root and open the notebooks in numerical order. Their configuration cells and saved outputs document the parameters relevant to each experiment. More recent object-oriented simulation work, including an explicit author entity, is maintained separately in the Readersourcing_OO repository[23] and underpins the evaluation by Soprano et al. [11]; it is a scientific extension, not an implicit change to the deployed RS_Server domain.

7 Overview

An overview of Readersourcing 2.0 capabilities is shown in Figure 16. Let us suppose that there are four readers using RS_Rate to rate a publication, `P1`, namely `RD1`, `RD2`, `RD3`, and `RD4`. Both `RD1`, `RD2`, and `RD3` utilize the *Save for later* functionality of RS_Rate to express their ratings at a later time. By doing this, they receive a link-annotated version of `P1`, namely `P1+Link`.

After some time, `RD1` opens `P1+Link` with a preferred PDF reader. `RD2` sends it to a tablet, while `RD3` opens it in a desktop browser. By following the link or scanning the QR code added through RS_PDF, they reach the dedicated RS_Server page and provide their ratings. In contrast, `RD4` rates `P1` immediately through the RS_Rate interface.

[23] `https://github.com/EddyMaddalena/Readersourcing_OO`

References

[1] Luca De Alfaro and Marco Faella. TrueReview: A Platform for Post-Publication Peer Review. In: *CoRR* abs/1608.07878 (2016). arXiv: 1608.07878. URL: http://arxiv.org/abs/1608.07878.

[2] Martin Fowler. *UML Distilled: A Brief Guide to the Standard Object Modeling Language*. 3rd ed. Boston, MA, USA: Addison-Wesley Longman Publishing Co., Inc., 2003. ISBN: 0321193687.

[3] Erich Gamma et al. *Design Patterns: Elementi per il Riuso di Software ad Oggetti*. Pearson, Jan. 2002.

[4] Stefano Mizzaro. Quality control in scholarly publishing: A new proposal. In: *Journal of the American Society for Information Science and Technology* 54.11 (2003), pp. 989–1005. DOI: 10.1002/asi.10296. URL: https://onlinelibrary.wiley.com/doi/abs/10.1002/asi.10296.

[5] Michael Soprano and Stefano Mizzaro. Crowdsourcing Peer Review: As We May Do. In: *Digital Libraries: Supporting Open Science*. Vol. 988. Communications in Computer and Information Science. Springer, 2019, pp. 259–273. DOI: 10.1007/978-3-030-11226-4_21. URL: https://doi.org/10.1007/978-3-030-11226-4_21.

[6] Michael Soprano and Stefano Mizzaro. *Readersourcing 2.0: RS_PDF*. DOI: 10.5281/zenodo.1442597. URL: https://doi.org/10.5281/zenodo.1442597.

[7] Michael Soprano and Stefano Mizzaro. *Readersourcing 2.0: RS_Py*. DOI: 10.5281/zenodo.3245208. URL: https://doi.org/10.5281/zenodo.3245208.

[8] Michael Soprano and Stefano Mizzaro. *Readersourcing 2.0: RS_Rate*. DOI: 10.5281/zenodo.1442599. URL: https://doi.org/10.5281/zenodo.1442599.

[9] Michael Soprano and Stefano Mizzaro. *Readersourcing 2.0: RS_Server*. DOI: 10.5281/zenodo.1442630. URL: https://doi.org/10.5281/zenodo.1442630.

[10] Michael Soprano, Kevin Roitero, and Stefano Mizzaro. HITS Hits Readersourcing: Validating Peer Review Alternatives Using Network Analysis. In: *Proceedings of the 4th Joint Workshop on Bibliometric-enhanced Information Retrieval and Natural Language Processing for Digital Libraries*. Vol. 2414. CEUR Workshop Proceedings. 2019, pp. 70–82. URL: https://ceur-ws.org/Vol-2414/paper7.pdf.

[11] Michael Soprano et al. Evaluation of Crowdsourced Peer Review using Synthetic Data and Simulations. In: *Proceedings of the 21st Conference on Information and Research Science Connecting to Digital and Library Science*. Vol. 3937. CEUR Workshop Proceedings. 2025. URL: https://ceur-ws.org/Vol-3937/paper8.pdf.